

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import scipy
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import mean_squared_error
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RandomizedSearchCV
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import LabelEncoder
from sklearn import tree
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
```

```
In [3]: # Importing the dataframe
strava_data = pd.read_csv("strava_data (1).csv")
```

```
In [4]: # Displays the dataframe
strava_data
```

Out[4]:

	Unnamed: 0	id	external_id	upload_id	athlete	name
0	0	282363847	9A54FEB2-B831-404F-911C-AC747346D5CB	3.190523e+08	{'id': 8583263}	Afternoon Ride
1	1	289575461	4CDDC3BE-7EEB-4E42-A267-83CCF01030DF	3.273658e+08	{'id': 8583263}	Afternoon Ride
2	2	290085349	5FA2E7AC-6255-4598-AC63-CEFF43CB9B2D	3.279653e+08	{'id': 8583263}	Morning Ride
3	3	290188116	E070401A-54B9-40FD-BF9E-068E7C5252E5	3.280794e+08	{'id': 8583263}	Lunch Run
4	4	290272559	4F35938D-5C57-4E13-9D05-137946A8AA5B	3.281866e+08	{'id': 8583263}	Afternoon Ride
...	...	...	...	...	...	...
1086	1086	2499341454	garmin_push_3806613793	2.654715e+09	{'id': 8583263}	Morning Ride
1087	1087	2500152952	garmin_push_3807545540	2.655548e+09	{'id': 8583263}	Afternoon Ride
1088	1088	2507262227	garmin_push_3815882396	2.662830e+09	{'id': 8583263}	Morning Ride
1089	1089	2532931516	garmin_push_3845982255	2.689061e+09	{'id': 8583263}	Morning Ride
1090	1090	2533486678	garmin_push_3846715423	2.689640e+09	{'id': 8583263}	Afternoon Ride

1091 rows × 40 columns



In [5]:

```
# Generates statistical summaries for the data sets numerical features
strava_data.describe
```

```
Out[5]: <bound method NDFrame.describe of          Unnamed: 0          id
external_id \
0          0      282363847  9A54FEB2-B831-404F-911C-AC747346D5CB
1          1      289575461  4CDDC3BE-7EEB-4E42-A267-83CCF01030DF
2          2      290085349  5FA2E7AC-6255-4598-AC63-CEFF43CB9B2D
3          3      290188116  E070401A-54B9-40FD-BF9E-068E7C5252E5
4          4      290272559  4F35938D-5C57-4E13-9D05-137946A8AA5B
...          ...          ...
1086        1086  2499341454          garmin_push_3806613793
1087        1087  2500152952          garmin_push_3807545540
1088        1088  2507262227          garmin_push_3815882396
1089        1089  2532931516          garmin_push_3845982255
1090        1090  2533486678          garmin_push_3846715423
```

```
          upload_id          athlete          name  distance  moving_time \
0      3.190523e+08 {'id': 8583263} Afternoon Ride    9789.1    1898
1      3.273658e+08 {'id': 8583263} Afternoon Ride    9967.4    1840
2      3.279653e+08 {'id': 8583263} Morning Ride   10022.0    1763
3      3.280794e+08 {'id': 8583263} Lunch Run     5227.2    1806
4      3.281866e+08 {'id': 8583263} Afternoon Ride    9969.3    1859
...          ...          ...          ...          ...          ...
1086  2.654715e+09 {'id': 8583263} Morning Ride   10283.8    1652
1087  2.655548e+09 {'id': 8583263} Afternoon Ride    9826.1    1742
1088  2.662830e+09 {'id': 8583263} Morning Ride   39854.6    8053
1089  2.689061e+09 {'id': 8583263} Morning Ride   10274.9    1689
1090  2.689640e+09 {'id': 8583263} Afternoon Ride    9853.7    1675
```

```
          elapsed_time  total_elevation_gain  ...  workout_type  average_speed \
0          2057          27.1  ...          0.0          5.158
1          2120          33.1  ...          0.0          5.417
2          1958          27.2  ...          0.0          5.685
3          1815          10.9  ...          0.0          2.894
4          2017          46.1  ...          0.0          5.363
...          ...          ...  ...          ...          ...
1086        1942          26.0  ...          NaN          6.225
1087        2034          42.0  ...          NaN          5.641
1088        9362         621.0  ...          NaN          4.949
1089        2213          27.0  ...          NaN          6.083
1090        1764          45.0  ...          NaN          5.883
```

```
          max_speed  has_kudoed  gear_id  kilojoules  average_watts  device_watts \
0          10.3        False        NaN        NaN        NaN        False
1          15.2        False        NaN        NaN        NaN        False
2          15.6        False        NaN        NaN        NaN        False
3           6.2        False        NaN        NaN        NaN        NaN
4          14.9        False        NaN       187.0       100.6        False
...          ...          ...          ...          ...          ...
1086          9.9        False  b3900175       115.7        70.0        False
1087          9.8        False  b3900175       139.7        80.2        False
1088         16.1        False  b3900175       820.5       101.9        False
1089          9.2        False  b3900175       114.0        67.5        False
1090         10.3        False  b3900175       112.9        67.4        False
```

```
          max_watts  weighted_average_watts
0          NaN          NaN
1          NaN          NaN
```

2	NaN	NaN
3	NaN	NaN
4	NaN	NaN
...	...	...
1086	NaN	NaN
1087	NaN	NaN
1088	NaN	NaN
1089	NaN	NaN
1090	NaN	NaN

[1091 rows x 40 columns]>

```
In [6]: # Computes a correlation maxrix showing how strongly each numeric variable is relat
strava_data.corr(numeric_only = True)
```

Out[6]:

	Unnamed: 0	id	upload_id	distance	moving_time	elapsed_time
Unnamed: 0	1.000000	0.997570	0.996807	0.267524	0.276598	0.173959
id	0.997570	1.000000	0.999925	0.266011	0.275198	0.169241
upload_id	0.996807	0.999925	1.000000	0.266783	0.275835	0.169872
distance	0.267524	0.266011	0.266783	1.000000	0.977939	0.602422
moving_time	0.276598	0.275198	0.275835	0.977939	1.000000	0.599641
elapsed_time	0.173959	0.169241	0.169872	0.602422	0.599641	1.000000
total_elevation_gain	0.261754	0.258591	0.258972	0.934715	0.960029	0.531935
elev_high	0.303989	0.300401	0.300122	0.713587	0.727764	0.562388
elev_low	0.123985	0.113936	0.112557	-0.015419	0.074609	0.014549
achievement_count	0.035108	0.037130	0.038567	0.781548	0.743475	0.420570
kudos_count	0.756961	0.746384	0.744727	0.663103	0.675752	0.411893
comment_count	0.143836	0.143504	0.144913	0.588138	0.594897	0.310920
athlete_count	0.117209	0.112356	0.112072	0.478289	0.435782	0.231807
photo_count	NaN	NaN	NaN	NaN	NaN	NaN
total_photo_count	0.179010	0.174495	0.175061	0.710554	0.719764	0.406230
trainer	-0.043834	-0.036277	-0.034838	-0.072784	-0.025926	-0.020230
commute	-0.075654	-0.064058	-0.062529	-0.040545	-0.041708	-0.031807
manual	0.005770	0.004699	NaN	-0.020268	-0.014119	-0.010130
private	NaN	NaN	NaN	NaN	NaN	NaN
flagged	NaN	NaN	NaN	NaN	NaN	NaN
workout_type	0.492304	0.504509	0.507885	0.323032	0.302764	0.271807
average_speed	0.077474	0.078800	0.079318	0.088263	-0.059051	0.020230
max_speed	-0.035103	-0.053091	-0.056559	0.507798	0.454597	0.281807
has_kudoed	NaN	NaN	NaN	NaN	NaN	NaN
kilojoules	0.222986	0.220006	0.220415	0.981158	0.985610	0.562388
average_watts	-0.261314	-0.274885	-0.277327	0.373944	0.352478	0.120570
max_watts	NaN	NaN	NaN	NaN	NaN	NaN
weighted_average_watts	NaN	NaN	NaN	NaN	NaN	NaN

28 rows × 28 columns




```
In [98]: # Creates a new column that makes the value equal to 0 if the max speed is less than 12.168
# 1 if the max speed is greater than 12.168
```

```
strava_data["speed_category"] = strava_data["max_speed"].apply(lambda x: 0 if x <=
```

```
In [109]: # Selects the target variable (speed category)
y = strava_data["speed_category"]
# Selects the feature columns used for prediction
X = strava_data[["distance", "moving_time", "total_elevation_gain", "achievement_co

# Splits the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2)
```

```
In [110]: # Creates a Decision Tree classifier
clf = tree.DecisionTreeClassifier()
# Trains the classifier using the training data
clf = clf.fit(X_train , y_train)
```

```
In [111]: # Uses the trained classifier to predict the target values for the training data
clf.predict(X_train)
```

```
Out[111...] array([1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1,
      0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0,
      1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0,
      0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1,
      1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
      0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0,
      0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0,
      1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1,
      1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0,
      0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1,
      0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1,
      0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0,
      1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0,
      0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0,
      1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0,
      0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1,
      1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0,
      1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0,
      1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0,
      1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
      0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1,
      1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0,
      0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1,
      1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0,
      1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0,
      1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0,
      0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0,
      1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0,
      1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0])
```

```
In [112...] # Predicts the target values for the test data using the trained classifier
y_pred = clf.predict(X_test)
```

```
In [113...] # Calculates the accuracy of the predictions by comparing them to the actual test L
accuracy = accuracy_score(y_test, y_pred)
# Prints the accuracy score
print("Accuracy:", accuracy)
```

Accuracy: 0.8538812785388128

```
In [120...] # Visualizes the structure of the trained decision tree
tree.plot_tree(clf)
```

```

Out[120... [Text(0.6863627373417721, 0.96875, 'x[0] <= 22314.8\ngini = 0.489\nsamples = 872\n
value = [501, 371]'),
Text(0.4064807489451477, 0.90625, 'x[0] <= 9306.25\ngini = 0.398\nsamples = 682\n
value = [495.0, 187.0]'),
Text(0.5464217431434599, 0.9375, 'True '),
Text(0.14029535864978904, 0.84375, 'x[0] <= 7588.3\ngini = 0.5\nsamples = 224\nva
lue = [112, 112]'),
Text(0.0379746835443038, 0.78125, 'x[0] <= 4252.05\ngini = 0.204\nsamples = 78\nv
alue = [69, 9]'),
Text(0.029535864978902954, 0.71875, 'gini = 0.0\nsamples = 34\nvalue = [34, 0]'),
Text(0.046413502109704644, 0.71875, 'x[1] <= 856.5\ngini = 0.325\nsamples = 44\nv
alue = [35, 9]'),
Text(0.016877637130801686, 0.65625, 'x[2] <= 26.0\ngini = 0.408\nsamples = 7\nval
ue = [2, 5]'),
Text(0.008438818565400843, 0.59375, 'gini = 0.0\nsamples = 4\nvalue = [0, 4]'),
Text(0.02531645569620253, 0.59375, 'x[0] <= 4325.6\ngini = 0.444\nsamples = 3\nva
lue = [2, 1]'),
Text(0.016877637130801686, 0.53125, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.03375527426160337, 0.53125, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
Text(0.0759493670886076, 0.65625, 'x[1] <= 1070.5\ngini = 0.193\nsamples = 37\nva
lue = [33, 4]'),
Text(0.05907172995780591, 0.59375, 'x[2] <= 22.8\ngini = 0.42\nsamples = 10\nvalu
e = [7, 3]'),
Text(0.05063291139240506, 0.53125, 'x[2] <= 11.95\ngini = 0.5\nsamples = 6\nvalue
= [3, 3]'),
Text(0.04219409282700422, 0.46875, 'x[0] <= 6530.65\ngini = 0.375\nsamples = 4\nv
alue = [3, 1]'),
Text(0.03375527426160337, 0.40625, 'gini = 0.0\nsamples = 3\nvalue = [3, 0]'),
Text(0.05063291139240506, 0.40625, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.05907172995780591, 0.46875, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.06751054852320675, 0.53125, 'gini = 0.0\nsamples = 4\nvalue = [4, 0]'),
Text(0.09282700421940929, 0.59375, 'x[3] <= 8.5\ngini = 0.071\nsamples = 27\nvalu
e = [26, 1]'),
Text(0.08438818565400844, 0.53125, 'gini = 0.0\nsamples = 23\nvalue = [23, 0]'),
Text(0.10126582278481013, 0.53125, 'x[2] <= 11.8\ngini = 0.375\nsamples = 4\nvalu
e = [3, 1]'),
Text(0.09282700421940929, 0.46875, 'x[0] <= 5534.55\ngini = 0.5\nsamples = 2\nval
ue = [1, 1]'),
Text(0.08438818565400844, 0.40625, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.10126582278481013, 0.40625, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.10970464135021098, 0.46875, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
Text(0.24261603375527427, 0.78125, 'x[1] <= 1754.5\ngini = 0.416\nsamples = 146\n
value = [43, 103]'),
Text(0.23417721518987342, 0.71875, 'x[2] <= 40.55\ngini = 0.384\nsamples = 139\nv
alue = [36.0, 103.0]'),
Text(0.1729957805907173, 0.65625, 'x[2] <= 31.1\ngini = 0.259\nsamples = 85\nvalu
e = [13.0, 72.0]'),
Text(0.14345991561181434, 0.59375, 'x[3] <= 2.5\ngini = 0.499\nsamples = 21\nvalu
e = [11, 10]'),
Text(0.1350210970464135, 0.53125, 'x[0] <= 7867.2\ngini = 0.475\nsamples = 18\nva
lue = [11, 7]'),
Text(0.12658227848101267, 0.46875, 'gini = 0.0\nsamples = 3\nvalue = [0, 3]'),
Text(0.14345991561181434, 0.46875, 'x[1] <= 1340.5\ngini = 0.391\nsamples = 15\nv
alue = [11, 4]'),
Text(0.12658227848101267, 0.40625, 'x[3] <= 0.5\ngini = 0.444\nsamples = 3\nvalue
= [1, 2]'),

```

Text(0.11814345991561181, 0.34375, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.1350210970464135, 0.34375, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.16033755274261605, 0.40625, 'x[1] <= 1694.0\ngini = 0.278\nsamples = 12\nvalue = [10, 2]'),
Text(0.1518987341772152, 0.34375, 'x[0] <= 8138.85\ngini = 0.165\nsamples = 11\nvalue = [10, 1]'),
Text(0.14345991561181434, 0.28125, 'x[3] <= 1.0\ngini = 0.5\nsamples = 2\nvalue = [1, 1]'),
Text(0.1350210970464135, 0.21875, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.1518987341772152, 0.21875, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.16033755274261605, 0.28125, 'gini = 0.0\nsamples = 9\nvalue = [9, 0]'),
Text(0.16877637130801687, 0.34375, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.1518987341772152, 0.53125, 'gini = 0.0\nsamples = 3\nvalue = [0, 3]'),
Text(0.20253164556962025, 0.59375, 'x[3] <= 2.5\ngini = 0.061\nsamples = 64\nvalue = [2, 62]'),
Text(0.1940928270042194, 0.53125, 'gini = 0.0\nsamples = 53\nvalue = [0, 53]'),
Text(0.2109704641350211, 0.53125, 'x[3] <= 3.5\ngini = 0.298\nsamples = 11\nvalue = [2, 9]'),
Text(0.20253164556962025, 0.46875, 'x[2] <= 35.15\ngini = 0.444\nsamples = 6\nvalue = [2, 4]'),
Text(0.1940928270042194, 0.40625, 'x[2] <= 33.65\ngini = 0.5\nsamples = 4\nvalue = [2, 2]'),
Text(0.18565400843881857, 0.34375, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.20253164556962025, 0.34375, 'x[1] <= 1364.5\ngini = 0.444\nsamples = 3\nvalue = [2, 1]'),
Text(0.1940928270042194, 0.28125, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
Text(0.2109704641350211, 0.28125, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.2109704641350211, 0.40625, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.21940928270042195, 0.46875, 'gini = 0.0\nsamples = 5\nvalue = [0, 5]'),
Text(0.29535864978902954, 0.65625, 'x[0] <= 8310.0\ngini = 0.489\nsamples = 54\nvalue = [23, 31]'),
Text(0.2616033755274262, 0.59375, 'x[3] <= 1.5\ngini = 0.463\nsamples = 22\nvalue = [14, 8]'),
Text(0.25316455696202533, 0.53125, 'x[1] <= 1257.0\ngini = 0.5\nsamples = 16\nvalue = [8, 8]'),
Text(0.24472573839662448, 0.46875, 'gini = 0.0\nsamples = 3\nvalue = [0, 3]'),
Text(0.2616033755274262, 0.46875, 'x[2] <= 43.85\ngini = 0.473\nsamples = 13\nvalue = [8, 5]'),
Text(0.25316455696202533, 0.40625, 'x[2] <= 40.7\ngini = 0.397\nsamples = 11\nvalue = [8, 3]'),
Text(0.23628691983122363, 0.34375, 'x[3] <= 0.5\ngini = 0.444\nsamples = 3\nvalue = [1, 2]'),
Text(0.22784810126582278, 0.28125, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.24472573839662448, 0.28125, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.270042194092827, 0.34375, 'x[0] <= 8135.95\ngini = 0.219\nsamples = 8\nvalue = [7, 1]'),
Text(0.2616033755274262, 0.28125, 'x[2] <= 42.0\ngini = 0.375\nsamples = 4\nvalue = [3, 1]'),
Text(0.25316455696202533, 0.21875, 'gini = 0.0\nsamples = 3\nvalue = [3, 0]'),
Text(0.270042194092827, 0.21875, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.27848101265822783, 0.28125, 'gini = 0.0\nsamples = 4\nvalue = [4, 0]'),
Text(0.270042194092827, 0.40625, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.270042194092827, 0.53125, 'gini = 0.0\nsamples = 6\nvalue = [6, 0]'),
Text(0.3291139240506329, 0.59375, 'x[1] <= 1373.0\ngini = 0.404\nsamples = 32\nvalue = [9, 23]'),
Text(0.31223628691983124, 0.53125, 'x[2] <= 65.3\ngini = 0.188\nsamples = 19\nvalue = [9, 23]')

```

ue = [2, 17]'),
  Text(0.3037974683544304, 0.46875, 'x[0] <= 8336.55\ngini = 0.105\nsamples = 18\nv
alue = [1, 17]'),
  Text(0.29535864978902954, 0.40625, 'x[0] <= 8325.5\ngini = 0.444\nsamples = 3\nva
alue = [1, 2]'),
  Text(0.2869198312236287, 0.34375, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
  Text(0.3037974683544304, 0.34375, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
  Text(0.31223628691983124, 0.40625, 'gini = 0.0\nsamples = 15\nvalue = [0, 15]'),
  Text(0.3206751054852321, 0.46875, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
  Text(0.3459915611814346, 0.53125, 'x[3] <= 2.0\ngini = 0.497\nsamples = 13\nvalue
= [7, 6]'),
  Text(0.33755274261603374, 0.46875, 'x[0] <= 8711.4\ngini = 0.42\nsamples = 10\nva
alue = [7, 3]'),
  Text(0.3291139240506329, 0.40625, 'x[1] <= 1401.0\ngini = 0.5\nsamples = 6\nvalue
= [3, 3]'),
  Text(0.3206751054852321, 0.34375, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
  Text(0.33755274261603374, 0.34375, 'x[1] <= 1520.5\ngini = 0.375\nsamples = 4\nva
alue = [1, 3]'),
  Text(0.3291139240506329, 0.28125, 'gini = 0.0\nsamples = 3\nvalue = [0, 3]'),
  Text(0.3459915611814346, 0.28125, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
  Text(0.3459915611814346, 0.40625, 'gini = 0.0\nsamples = 4\nvalue = [4, 0]'),
  Text(0.35443037974683544, 0.46875, 'gini = 0.0\nsamples = 3\nvalue = [0, 3]'),
  Text(0.2510548523206751, 0.71875, 'gini = 0.0\nsamples = 7\nvalue = [7, 0]'),
  Text(0.6726661392405063, 0.84375, 'x[3] <= 1.5\ngini = 0.274\nsamples = 458\nvalu
e = [383, 75]'),
  Text(0.5921677215189873, 0.78125, 'x[2] <= 68.45\ngini = 0.155\nsamples = 343\nva
alue = [314.0, 29.0]'),
  Text(0.5387658227848101, 0.71875, 'x[1] <= 2065.0\ngini = 0.102\nsamples = 317\nv
alue = [300, 17]'),
  Text(0.495253164556962, 0.65625, 'x[0] <= 11338.1\ngini = 0.087\nsamples = 309\nv
alue = [295.0, 14.0]'),
  Text(0.45042194092827004, 0.59375, 'x[3] <= 0.5\ngini = 0.076\nsamples = 305\nval
ue = [293.0, 12.0]'),
  Text(0.39451476793248946, 0.53125, 'x[1] <= 1481.0\ngini = 0.049\nsamples = 238\n
value = [232, 6]'),
  Text(0.37130801687763715, 0.46875, 'x[0] <= 9728.1\ngini = 0.5\nsamples = 2\nvalu
e = [1, 1]'),
  Text(0.3628691983122363, 0.40625, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
  Text(0.379746835443038, 0.40625, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
  Text(0.4177215189873418, 0.46875, 'x[1] <= 1561.5\ngini = 0.041\nsamples = 236\nv
alue = [231, 5]'),
  Text(0.39662447257383965, 0.40625, 'x[1] <= 1560.5\ngini = 0.165\nsamples = 22\nv
alue = [20, 2]'),
  Text(0.3881856540084388, 0.34375, 'x[0] <= 10179.85\ngini = 0.091\nsamples = 21\n
value = [20, 1]'),
  Text(0.379746835443038, 0.28125, 'gini = 0.0\nsamples = 19\nvalue = [19, 0]'),
  Text(0.39662447257383965, 0.28125, 'x[1] <= 1533.0\ngini = 0.5\nsamples = 2\nvalu
e = [1, 1]'),
  Text(0.3881856540084388, 0.21875, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
  Text(0.4050632911392405, 0.21875, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
  Text(0.4050632911392405, 0.34375, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
  Text(0.4388185654008439, 0.40625, 'x[0] <= 9599.2\ngini = 0.028\nsamples = 214\nv
alue = [211, 3]'),
  Text(0.4219409282700422, 0.34375, 'x[1] <= 1714.5\ngini = 0.32\nsamples = 5\nvalu
e = [4, 1]'),
  Text(0.41350210970464135, 0.28125, 'gini = 0.0\nsamples = 4\nvalue = [4, 0]'),

```

```

Text(0.43037974683544306, 0.28125, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.45569620253164556, 0.34375, 'x[2] <= 33.15\ngini = 0.019\nsamples = 209\nvalue = [207, 2]'),
Text(0.4472573839662447, 0.28125, 'x[2] <= 33.05\ngini = 0.036\nsamples = 109\nvalue = [107, 2]'),
Text(0.43037974683544306, 0.21875, 'x[1] <= 1648.0\ngini = 0.019\nsamples = 107\nvalue = [106.0, 1.0]'),
Text(0.4219409282700422, 0.15625, 'x[1] <= 1645.5\ngini = 0.071\nsamples = 27\nvalue = [26, 1]'),
Text(0.41350210970464135, 0.09375, 'gini = 0.0\nsamples = 26\nvalue = [26, 0]'),
Text(0.43037974683544306, 0.09375, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.4388185654008439, 0.15625, 'gini = 0.0\nsamples = 80\nvalue = [80, 0]'),
Text(0.4641350210970464, 0.21875, 'x[0] <= 10239.45\ngini = 0.5\nsamples = 2\nvalue = [1, 1]'),
Text(0.45569620253164556, 0.15625, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.47257383966244726, 0.15625, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.4641350210970464, 0.28125, 'gini = 0.0\nsamples = 100\nvalue = [100, 0]'),
Text(0.5063291139240507, 0.53125, 'x[1] <= 1807.5\ngini = 0.163\nsamples = 67\nvalue = [61, 6]'),
Text(0.48945147679324896, 0.46875, 'x[2] <= 29.5\ngini = 0.098\nsamples = 58\nvalue = [55, 3]'),
Text(0.4810126582278481, 0.40625, 'gini = 0.0\nsamples = 34\nvalue = [34, 0]'),
Text(0.4978902953586498, 0.40625, 'x[1] <= 1616.0\ngini = 0.219\nsamples = 24\nvalue = [21, 3]'),
Text(0.48945147679324896, 0.34375, 'x[2] <= 32.5\ngini = 0.42\nsamples = 10\nvalue = [7, 3]'),
Text(0.4810126582278481, 0.28125, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.4978902953586498, 0.28125, 'x[1] <= 1595.5\ngini = 0.219\nsamples = 8\nvalue = [7, 1]'),
Text(0.48945147679324896, 0.21875, 'gini = 0.0\nsamples = 7\nvalue = [7, 0]'),
Text(0.5063291139240507, 0.21875, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.5063291139240507, 0.34375, 'gini = 0.0\nsamples = 14\nvalue = [14, 0]'),
Text(0.5232067510548524, 0.46875, 'x[0] <= 10057.55\ngini = 0.444\nsamples = 9\nvalue = [6, 3]'),
Text(0.5147679324894515, 0.40625, 'gini = 0.0\nsamples = 4\nvalue = [4, 0]'),
Text(0.5316455696202531, 0.40625, 'x[2] <= 60.5\ngini = 0.48\nsamples = 5\nvalue = [2, 3]'),
Text(0.5232067510548524, 0.34375, 'gini = 0.0\nsamples = 3\nvalue = [0, 3]'),
Text(0.540084388185654, 0.34375, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
Text(0.540084388185654, 0.59375, 'x[2] <= 13.6\ngini = 0.5\nsamples = 4\nvalue = [2, 2]'),
Text(0.5316455696202531, 0.53125, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.5485232067510548, 0.53125, 'x[3] <= 0.5\ngini = 0.444\nsamples = 3\nvalue = [1, 2]'),
Text(0.540084388185654, 0.46875, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.5569620253164557, 0.46875, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.5822784810126582, 0.65625, 'x[0] <= 10671.45\ngini = 0.469\nsamples = 8\nvalue = [5, 3]'),
Text(0.5738396624472574, 0.59375, 'x[1] <= 2114.5\ngini = 0.375\nsamples = 4\nvalue = [1, 3]'),
Text(0.5654008438818565, 0.53125, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.5822784810126582, 0.53125, 'x[0] <= 10054.05\ngini = 0.5\nsamples = 2\nvalue = [1, 1]'),
Text(0.5738396624472574, 0.46875, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.5907172995780591, 0.46875, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.5907172995780591, 0.59375, 'gini = 0.0\nsamples = 4\nvalue = [4, 0]'),

```

Text(0.6455696202531646, 0.71875, 'x[1] <= 3343.0\ngini = 0.497\nsamples = 26\nvalue = [14, 12]'),
Text(0.6244725738396625, 0.65625, 'x[1] <= 1808.0\ngini = 0.484\nsamples = 17\nvalue = [7, 10]'),
Text(0.6075949367088608, 0.59375, 'x[1] <= 1606.0\ngini = 0.408\nsamples = 7\nvalue = [5, 2]'),
Text(0.5991561181434599, 0.53125, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.6160337552742616, 0.53125, 'gini = 0.0\nsamples = 5\nvalue = [5, 0]'),
Text(0.6413502109704642, 0.59375, 'x[0] <= 11342.3\ngini = 0.32\nsamples = 10\nvalue = [2, 8]'),
Text(0.6329113924050633, 0.53125, 'gini = 0.0\nsamples = 6\nvalue = [0, 6]'),
Text(0.6497890295358649, 0.53125, 'x[1] <= 2751.0\ngini = 0.5\nsamples = 4\nvalue = [2, 2]'),
Text(0.6413502109704642, 0.46875, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
Text(0.6582278481012658, 0.46875, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.6666666666666666, 0.65625, 'x[0] <= 20490.85\ngini = 0.346\nsamples = 9\nvalue = [7, 2]'),
Text(0.6582278481012658, 0.59375, 'gini = 0.0\nsamples = 6\nvalue = [6, 0]'),
Text(0.6751054852320675, 0.59375, 'x[1] <= 3458.0\ngini = 0.444\nsamples = 3\nvalue = [1, 2]'),
Text(0.6666666666666666, 0.53125, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.6835443037974683, 0.53125, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.7531645569620253, 0.78125, 'x[2] <= 23.0\ngini = 0.48\nsamples = 115\nvalue = [69, 46]'),
Text(0.7088607594936709, 0.71875, 'x[1] <= 1567.5\ngini = 0.087\nsamples = 22\nvalue = [21, 1]'),
Text(0.70042194092827, 0.65625, 'x[1] <= 1553.5\ngini = 0.375\nsamples = 4\nvalue = [3, 1]'),
Text(0.6919831223628692, 0.59375, 'gini = 0.0\nsamples = 3\nvalue = [3, 0]'),
Text(0.7088607594936709, 0.59375, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.7172995780590717, 0.65625, 'gini = 0.0\nsamples = 18\nvalue = [18, 0]'),
Text(0.7974683544303798, 0.71875, 'x[2] <= 57.55\ngini = 0.499\nsamples = 93\nvalue = [48, 45]'),
Text(0.7341772151898734, 0.65625, 'x[2] <= 55.5\ngini = 0.484\nsamples = 56\nvalue = [23, 33]'),
Text(0.7257383966244726, 0.59375, 'x[1] <= 1453.5\ngini = 0.495\nsamples = 51\nvalue = [23.0, 28.0]'),
Text(0.7172995780590717, 0.53125, 'gini = 0.0\nsamples = 4\nvalue = [4, 0]'),
Text(0.7341772151898734, 0.53125, 'x[0] <= 10027.2\ngini = 0.482\nsamples = 47\nvalue = [19, 28]'),
Text(0.7130801687763713, 0.46875, 'x[0] <= 9801.85\ngini = 0.346\nsamples = 18\nvalue = [4, 14]'),
Text(0.7046413502109705, 0.40625, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
Text(0.7215189873417721, 0.40625, 'x[0] <= 9920.75\ngini = 0.219\nsamples = 16\nvalue = [2, 14]'),
Text(0.7130801687763713, 0.34375, 'x[3] <= 2.5\ngini = 0.444\nsamples = 6\nvalue = [2, 4]'),
Text(0.7046413502109705, 0.28125, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.7215189873417721, 0.28125, 'x[2] <= 26.5\ngini = 0.32\nsamples = 5\nvalue = [1, 4]'),
Text(0.7130801687763713, 0.21875, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.729957805907173, 0.21875, 'gini = 0.0\nsamples = 4\nvalue = [0, 4]'),
Text(0.729957805907173, 0.34375, 'gini = 0.0\nsamples = 10\nvalue = [0, 10]'),
Text(0.7552742616033755, 0.46875, 'x[1] <= 1577.0\ngini = 0.499\nsamples = 29\nvalue = [15.0, 14.0]'),
Text(0.7468354430379747, 0.40625, 'gini = 0.0\nsamples = 7\nvalue = [0, 7]'),

```

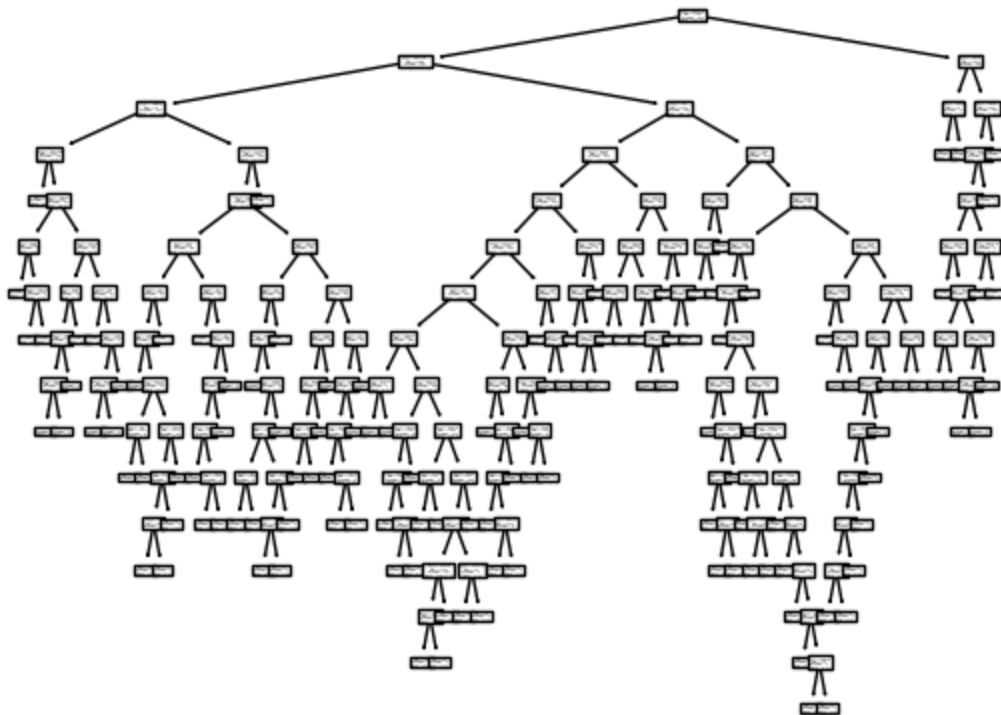
Text(0.7637130801687764, 0.40625, 'x[0] <= 10617.75\ngini = 0.434\nsamples = 22\nvalue = [15, 0, 7, 0]'),
Text(0.7468354430379747, 0.34375, 'x[0] <= 10142.55\ngini = 0.153\nsamples = 12\nvalue = [11, 1]'),
Text(0.7383966244725738, 0.28125, 'gini = 0.0\nsamples = 8\nvalue = [8, 0]'),
Text(0.7552742616033755, 0.28125, 'x[0] <= 10179.95\ngini = 0.375\nsamples = 4\nvalue = [3, 1]'),
Text(0.7468354430379747, 0.21875, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.7637130801687764, 0.21875, 'gini = 0.0\nsamples = 3\nvalue = [3, 0]'),
Text(0.7805907172995781, 0.34375, 'x[0] <= 11002.3\ngini = 0.48\nsamples = 10\nvalue = [4, 6]'),
Text(0.7721518987341772, 0.28125, 'gini = 0.0\nsamples = 3\nvalue = [0, 3]'),
Text(0.7890295358649789, 0.28125, 'x[2] <= 31.1\ngini = 0.49\nsamples = 7\nvalue = [4, 3]'),
Text(0.7805907172995781, 0.21875, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
Text(0.7974683544303798, 0.21875, 'x[2] <= 45.2\ngini = 0.48\nsamples = 5\nvalue = [2, 3]'),
Text(0.7890295358649789, 0.15625, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.8059071729957806, 0.15625, 'x[2] <= 49.7\ngini = 0.444\nsamples = 3\nvalue = [2, 1]'),
Text(0.7974683544303798, 0.09375, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.8143459915611815, 0.09375, 'x[1] <= 1884.5\ngini = 0.5\nsamples = 2\nvalue = [1, 1]'),
Text(0.8059071729957806, 0.03125, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.8227848101265823, 0.03125, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.7426160337552743, 0.59375, 'gini = 0.0\nsamples = 5\nvalue = [0, 5]'),
Text(0.8607594936708861, 0.65625, 'x[2] <= 67.0\ngini = 0.438\nsamples = 37\nvalue = [25, 12]'),
Text(0.8312236286919831, 0.59375, 'x[1] <= 1794.5\ngini = 0.208\nsamples = 17\nvalue = [15, 2]'),
Text(0.8227848101265823, 0.53125, 'gini = 0.0\nsamples = 10\nvalue = [10, 0]'),
Text(0.8396624472573839, 0.53125, 'x[1] <= 1899.5\ngini = 0.408\nsamples = 7\nvalue = [5, 2]'),
Text(0.8312236286919831, 0.46875, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.8481012658227848, 0.46875, 'gini = 0.0\nsamples = 5\nvalue = [5, 0]'),
Text(0.890295358649789, 0.59375, 'x[0] <= 20096.551\ngini = 0.5\nsamples = 20\nvalue = [10, 10]'),
Text(0.8734177215189873, 0.53125, 'x[3] <= 4.0\ngini = 0.459\nsamples = 14\nvalue = [5, 9]'),
Text(0.8649789029535865, 0.46875, 'x[2] <= 107.0\ngini = 0.469\nsamples = 8\nvalue = [5, 3]'),
Text(0.8565400843881856, 0.40625, 'x[0] <= 14019.75\ngini = 0.408\nsamples = 7\nvalue = [5, 2]'),
Text(0.8481012658227848, 0.34375, 'x[0] <= 13241.25\ngini = 0.48\nsamples = 5\nvalue = [3, 2]'),
Text(0.8396624472573839, 0.28125, 'x[2] <= 70.5\ngini = 0.375\nsamples = 4\nvalue = [3, 1]'),
Text(0.8312236286919831, 0.21875, 'x[1] <= 1796.0\ngini = 0.5\nsamples = 2\nvalue = [1, 1]'),
Text(0.8227848101265823, 0.15625, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.8396624472573839, 0.15625, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.8481012658227848, 0.21875, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
Text(0.8565400843881856, 0.28125, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.8649789029535865, 0.34375, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
Text(0.8734177215189873, 0.40625, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.8818565400843882, 0.46875, 'gini = 0.0\nsamples = 6\nvalue = [0, 6]'),

```

```

Text(0.9071729957805907, 0.53125, 'x[2] <= 89.5\ngini = 0.278\nsamples = 6\nvalue
= [5, 1]'),
Text(0.8987341772151899, 0.46875, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.9156118143459916, 0.46875, 'gini = 0.0\nsamples = 5\nvalue = [5, 0]'),
Text(0.9662447257383966, 0.90625, 'x[2] <= 157.5\ngini = 0.061\nsamples = 190\nva
lue = [6, 184]'),
Text(0.8263037315400843, 0.9375, ' False'),
Text(0.9493670886075949, 0.84375, 'x[2] <= 122.5\ngini = 0.5\nsamples = 4\nvalue
= [2, 2]'),
Text(0.9409282700421941, 0.78125, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
Text(0.9578059071729957, 0.78125, 'gini = 0.0\nsamples = 2\nvalue = [2, 0]'),
Text(0.9831223628691983, 0.84375, 'x[2] <= 732.0\ngini = 0.042\nsamples = 186\nva
lue = [4, 182]'),
Text(0.9746835443037974, 0.78125, 'x[0] <= 69592.25\ngini = 0.071\nsamples = 109
\nvalue = [4, 105]'),
Text(0.9662447257383966, 0.71875, 'x[2] <= 722.0\ngini = 0.054\nsamples = 108\nva
lue = [3, 105]'),
Text(0.9493670886075949, 0.65625, 'x[1] <= 6078.5\ngini = 0.037\nsamples = 106\nv
alue = [2, 104]'),
Text(0.9409282700421941, 0.59375, 'gini = 0.0\nsamples = 53\nvalue = [0, 53]'),
Text(0.9578059071729957, 0.59375, 'x[1] <= 6119.0\ngini = 0.073\nsamples = 53\nva
lue = [2, 51]'),
Text(0.9409282700421941, 0.53125, 'x[3] <= 10.0\ngini = 0.5\nsamples = 2\nvalue =
[1, 1]'),
Text(0.9324894514767933, 0.46875, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.9493670886075949, 0.46875, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.9746835443037974, 0.53125, 'x[0] <= 30387.45\ngini = 0.038\nsamples = 51\nv
alue = [1, 50]'),
Text(0.9662447257383966, 0.46875, 'x[1] <= 6461.5\ngini = 0.5\nsamples = 2\nvalue
= [1, 1]'),
Text(0.9578059071729957, 0.40625, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.9746835443037974, 0.40625, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.9831223628691983, 0.46875, 'gini = 0.0\nsamples = 49\nvalue = [0, 49]'),
Text(0.9831223628691983, 0.65625, 'x[0] <= 45640.6\ngini = 0.5\nsamples = 2\nvalu
e = [1, 1]'),
Text(0.9746835443037974, 0.59375, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
Text(0.9915611814345991, 0.59375, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.9831223628691983, 0.71875, 'gini = 0.0\nsamples = 1\nvalue = [1, 0]'),
Text(0.9915611814345991, 0.78125, 'gini = 0.0\nsamples = 77\nvalue = [0, 77]')]

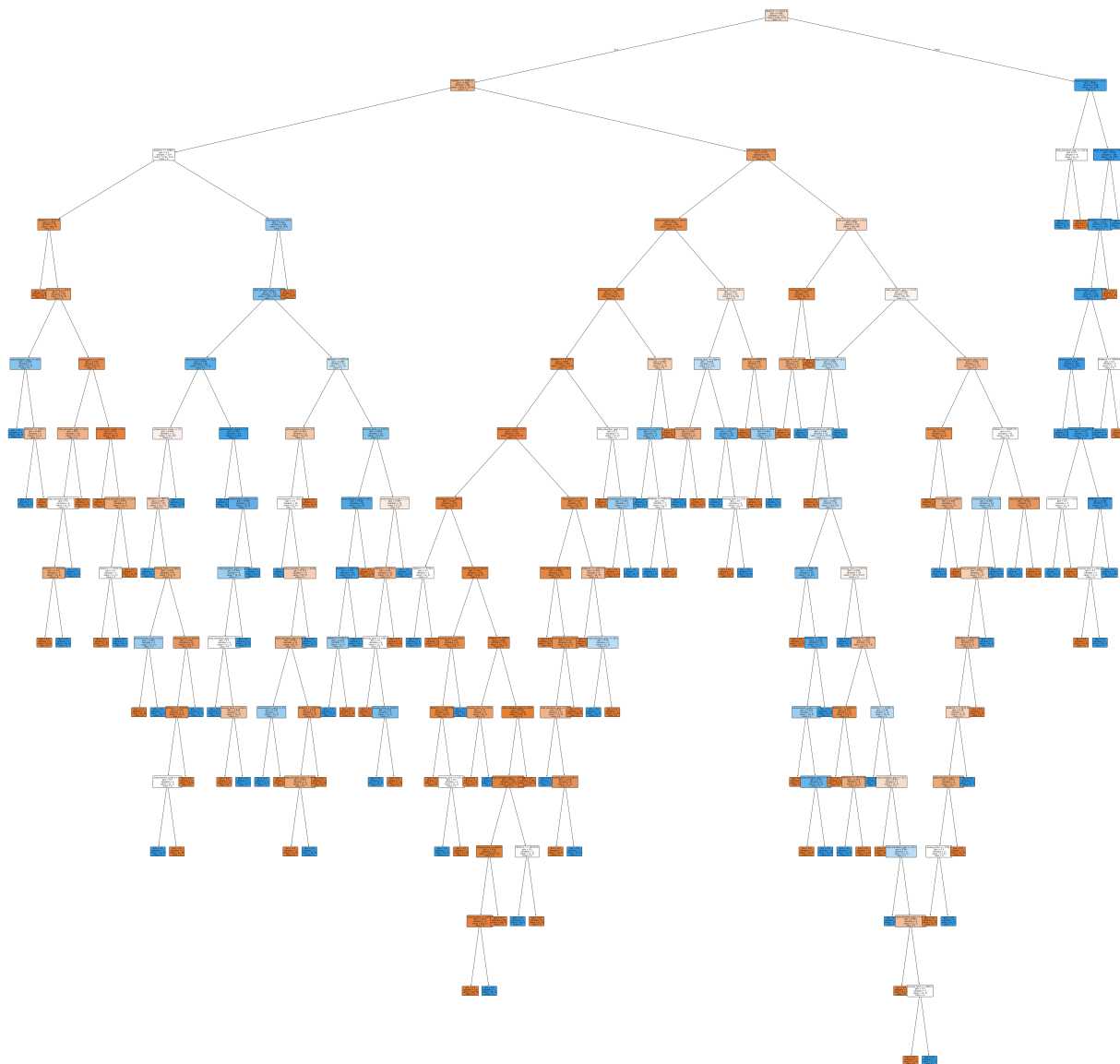
```



In [121...

```
# Sets the figure size for the tree plot
plt.figure(figsize=(80,80))
# Plots the decision tree with detailed formatting
tree.plot_tree(
    clf,                # The trained classifier
    filled=True,        # Fill nodes with colors based on class
    rounded=True,       # Round the corners of nodes
    feature_names=X_train.columns, # Show class names
    class_names=[str(c) for c in clf.classes_], # Set font size for readability
    fontsize=8
)

# Displays the plot
plt.show()
```

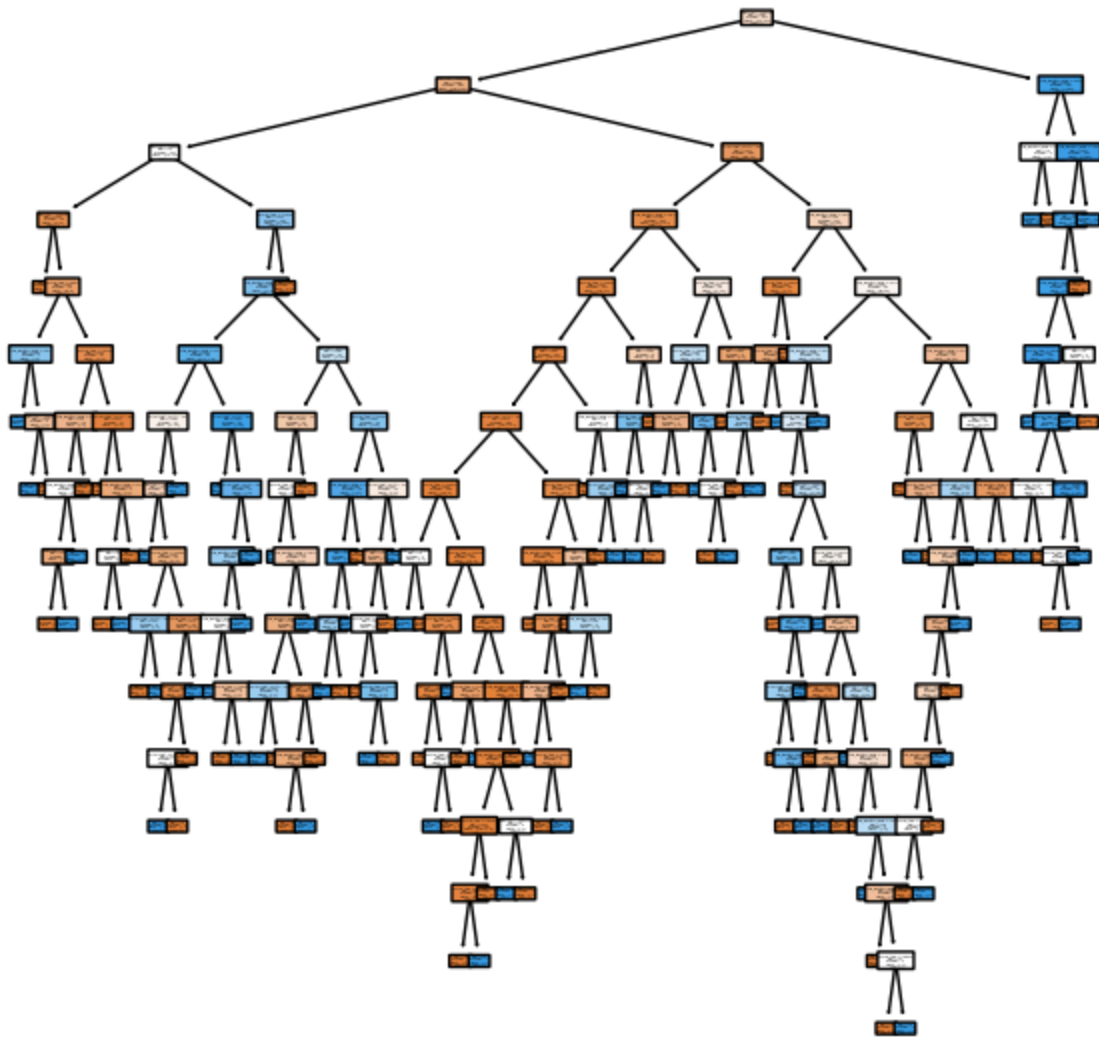


In [122...

```
# Sets the figure size for a smaller tree plot
plt.figure(figsize=(7,7))

# Plots the decision tree with detailed formatting for a smaller display
tree.plot_tree(
    clf,                                # The trained classifier
    filled=True,                        # Fill nodes with colors based on class
    rounded=True,                      # Round the corners of nodes
    feature_names=X_train.columns,     # Show class names
    class_names=[str(c) for c in clf.classes_], # Show class names
    fontsize=1
)

# Displays the plot
plt.show()
```



Regression

```
In [41]: # Selects the kudos_count column from the dataset
kudos_count = strava_data["kudos_count"]
```

```
# Selects the distance column from the dataset
distance = strava_data["distance"]
```

```
In [42]: # Performs a linear regression between kudos_count and distance
linear_model = scipy.stats.linregress(kudos_count, distance)
```

```
In [43]: # Displays the results of the linear regression (slope, intercept, r-value, p-value)
linear_model
```

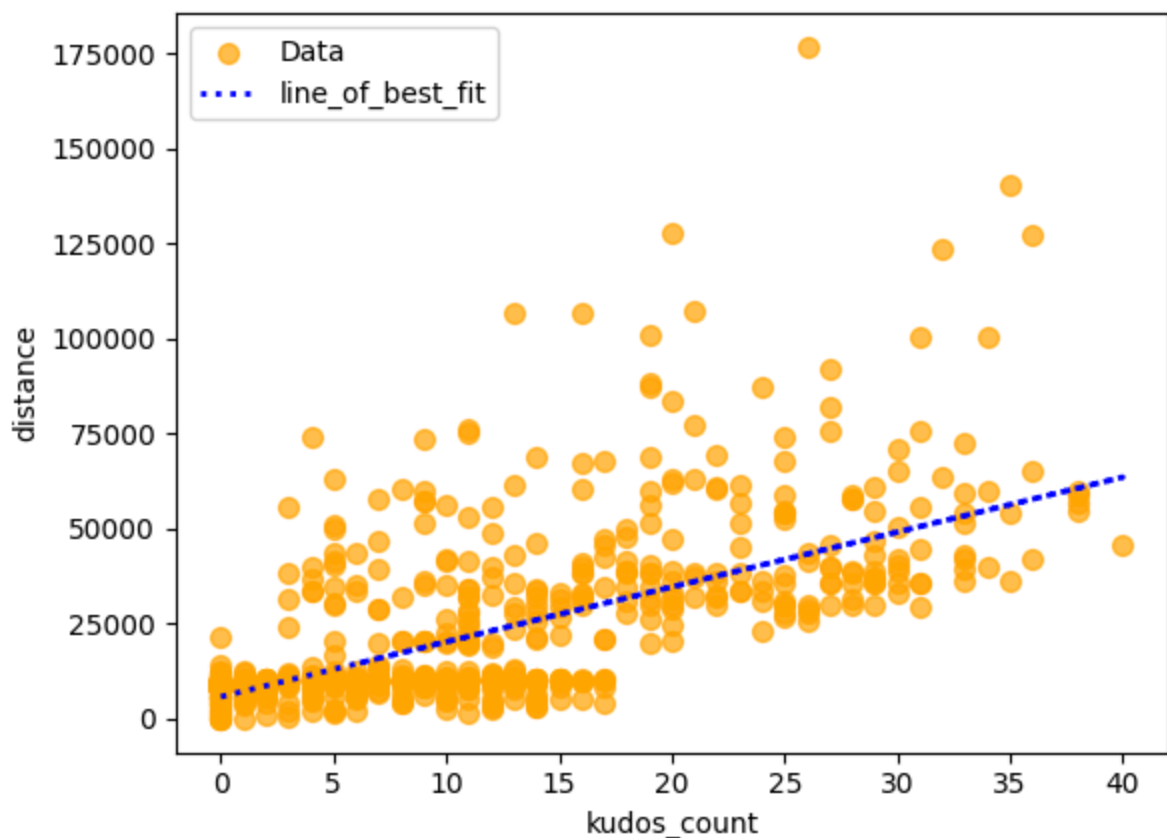
```
Out[43]: LinregressResult(slope=np.float64(1443.705087551276), intercept=np.float64(5737.57
03939333396), rvalue=np.float64(0.6631029233421656), pvalue=np.float64(3.7493231824
99183e-139), stderr=np.float64(49.384628286605206), intercept_stderr=np.float64(59
7.8657102770668))
```

```
In [44]: # Extracts the slope from the linear regression results
slope = linear_model.slope
# Extract the y-intercept from the linear regression results
y_intercept = linear_model.intercept

# Computes the predicted distance values (line of best fit) based on kudos_count
line_of_best_fit = slope*kudos_count+y_intercept
```

```
In [49]: # Creates a scatter plot of kudos_count vs distance
strava_data.plot.scatter("kudos_count","distance",s=50,c="orange", alpha=0.7, label
# Plots the line of best fit on top of the scatter plot
plt.plot(kudos_count, line_of_best_fit,linewidth = 2, color = "blue", linestyle = "
# Displays the legend in the upper-left corner
plt.legend(loc = "upper left")
```

Out[49]: <matplotlib.legend.Legend at 0x18bbccd6350>



```
In [50]: # Calculates the Pearson correlation coefficient and p-value between kudos_count and
corr = scipy.stats.pearsonr(strava_data["kudos_count"],strava_data["distance"])
```

```
In [51]: # Prints the Pearson correlation coefficient value
print(corr.statistic)
```

0.6631029233421648

```
In [53]: # Prints the slope of the linear regression line
print (slope)
```

1443.705087551276

```
In [5]: # Selects the kudos_count column from the dataset
kudos_count = strava_data["kudos_count"]
# Selects the moving_time column from the dataset
moving_time = strava_data["moving_time"]
```

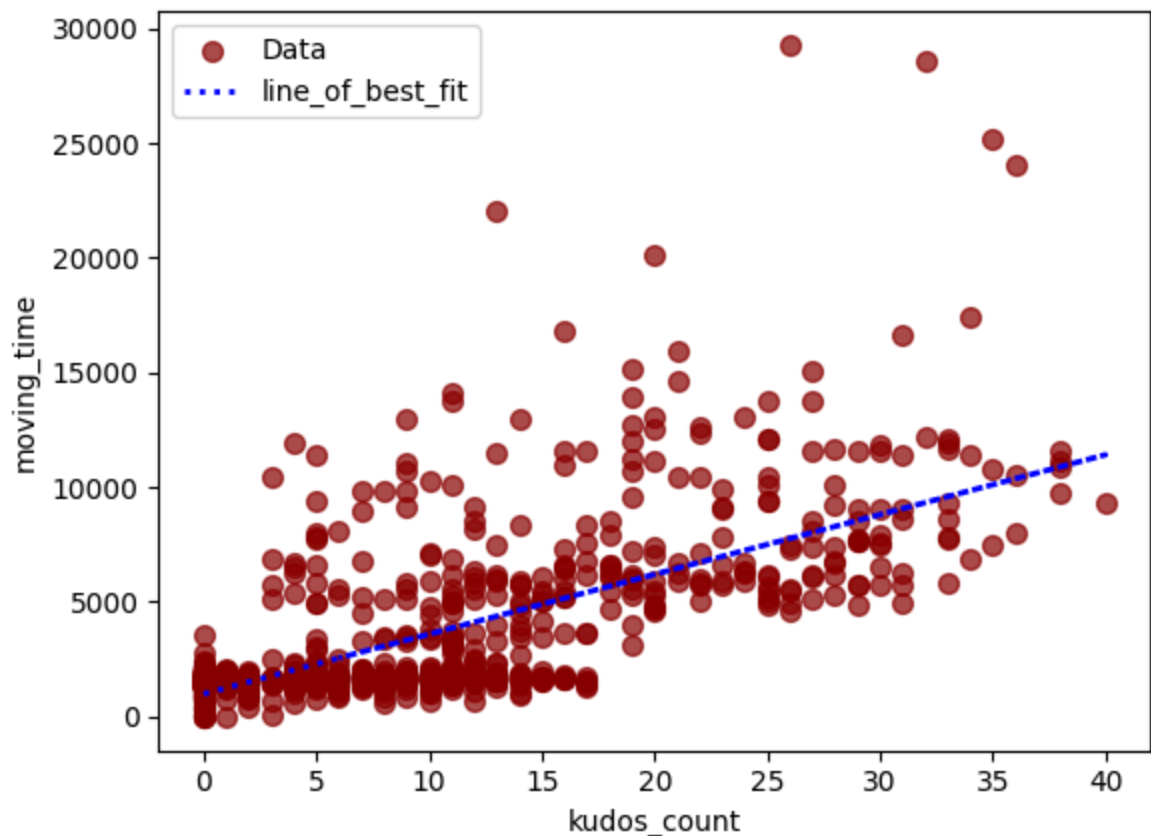
```
In [6]: # Performs a linear regression between kudos_count and moving_time
linear_model = scipy.stats.linregress(kudos_count, moving_time)
```

```
In [7]: # Extracts the slope from the linear regression results
slope = linear_model.slope
# Extracts the y-intercept from the linear regression results
y_intercept = linear_model.intercept

# Computes the predicted moving_time values (line of best fit) based on kudos_count
line_of_best_fit = slope*kudos_count+y_intercept
```

```
In [11]: # Create a scatter plot of kudos_count vs moving_time
strava_data.plot.scatter("kudos_count", "moving_time", s=50, c="darkred", alpha=0.7, 1
# Plots the line of best fit on top of the scatter plot
plt.plot(kudos_count, line_of_best_fit, linewidth = 2, color = "blue", linestyle = "
# Displays the legend in the upper-left corner
plt.legend(loc = "upper left")
```

Out[11]: <matplotlib.legend.Legend at 0x1acd251afd0>



```
In [12]: # Calculate the Pearson correlation coefficient and p-value between kudos_count and
corr = scipy.stats.pearsonr(strava_data["kudos_count"], strava_data["moving_time"])
```

```
In [14]: # Prints the Pearson correlation coefficient value
print(corr.statistic)
```

0.6757519570681574

```
In [19]: # Select the kudos_count column from the dataset
kudos_count = strava_data["kudos_count"]
# Select the total_elevation_gain column from the dataset
total_elevation_gain = strava_data["total_elevation_gain"]
```

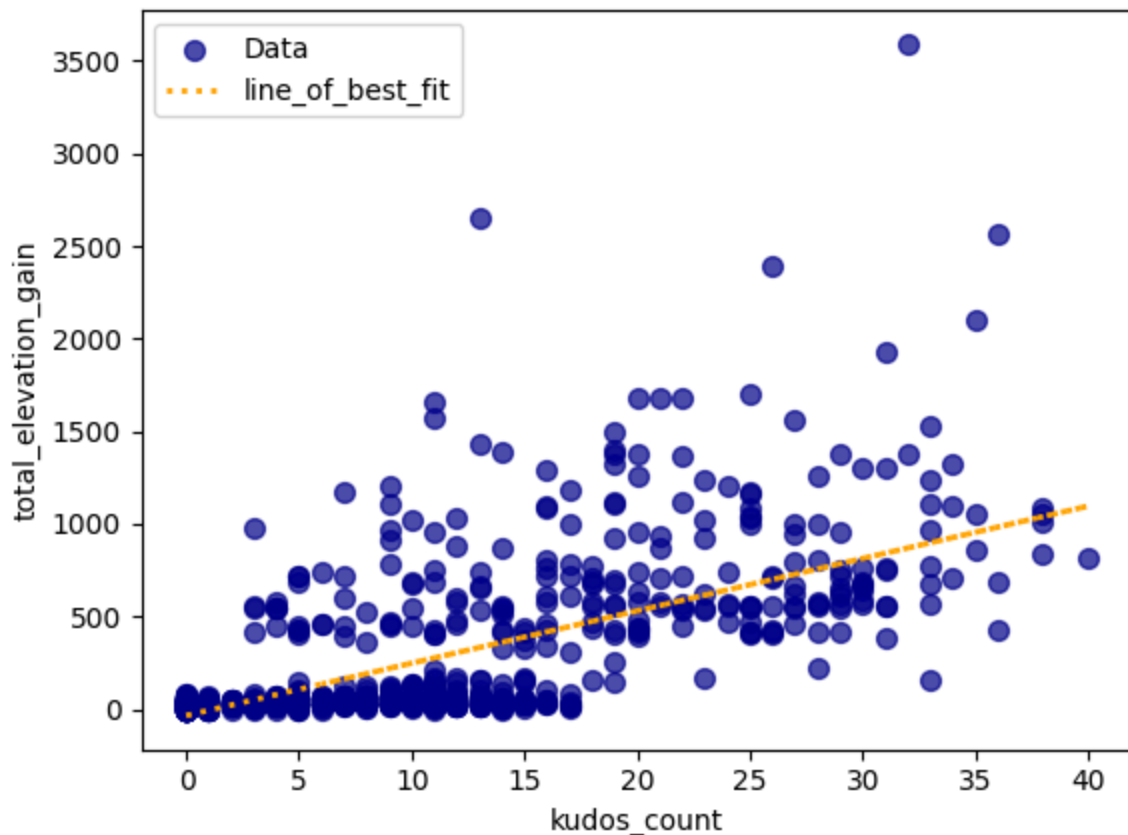
```
In [20]: # Performs a linear regression between kudos_count and total_elevation_gain
linear_model = scipy.stats.linregress(kudos_count, total_elevation_gain)
```

```
In [21]: # Extracts the slope from the linear regression results
slope = linear_model.slope
# Extracts the y-intercept from the linear regression results
y_intercept = linear_model.intercept

# Computes the predicted total_elevation_gain values (line of best fit) based on ku
line_of_best_fit = slope*kudos_count+y_intercept
```

```
In [27]: # Create a scatter plot of kudos_count vs total_elevation_gain
strava_data.plot.scatter("kudos_count", "total_elevation_gain", s=50, c="darkblue", al
# Plot the line of best fit on top of the scatter plot
plt.plot(kudos_count, line_of_best_fit, linewidth = 2, color = "orange", linestyle =
# Display the legend in the upper-left corner
plt.legend(loc = "upper left")
```

Out[27]: <matplotlib.legend.Legend at 0x1acd5933610>



```
In [28]: # Calculates the Pearson correlation coefficient and p-value between kudos_count and total_elevation_gain
corr = scipy.stats.pearsonr(strava_data["kudos_count"], strava_data["total_elevation_gain"])
```

```
In [29]: # Prints the Pearson correlation coefficient value
print(corr.statistic)
```

0.6648243746855161

```
In [10]: # Selects features and target variable
X = strava_data[["moving_time", "distance", "total_elevation_gain"]]
y = strava_data["kudos_count"]

# Standardizes the features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Splits the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Creates and trains a Multi-Layer Perceptron (MLP) regressor
mlp_regress = MLPRegressor(hidden_layer_sizes = [150, 100, 50, 25], activation = "relu")
# Trains the model
mlp_regress.fit(X_train, y_train)
# Predicts target values for the test set
y_pred = mlp_regress.predict(X_test)

# Calculates and prints the root mean squared error (RMSE) of the predictions
print("RMSE:", mean_squared_error(y_pred, y_test)**(0.5))
```

RMSE: 5.283594199664919

```
In [11]: # Creates a basic Multi-Layer Perceptron (MLP) regressor
mlp_regress = MLPRegressor()

# Define a set of hyperparameter options for random search
params_options = {
    "hidden_layer_sizes": [[10, 10], [25, 25], [50], [25]], # Different architectures
    "activation": ["logistic"], # Activation function
    "max_iter": [2500], # Maximum number of iterations
}
# Performs randomized search with cross-validation to find the best hyperparameters
random_search = RandomizedSearchCV(mlp_regress, params_options, n_iter = 20, scoring = "neg_mean_squared_error")
# Fit the random search on training data
random_search.fit(X_train, y_train)

# Prints the best hyperparameters found
print("Best Hyperparameters:", random_search.best_params_)
# Prints the best R² score achieved during cross-validation
print("Best R2 Value:", random_search.best_score_)
```

```

C:\Users\Abigail Miller\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_
qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\sklearn\model_select
ion\_search.py:317: UserWarning: The total space of parameters 4 is smaller than n_i
ter=20. Running 4 iterations. For exhaustive searches, use GridSearchCV.
  warnings.warn(
C:\Users\Abigail Miller\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_
qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\sklearn\neural_netwo
rk\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (2500) reached and the optimization hasn't converged yet.
  warnings.warn(
C:\Users\Abigail Miller\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_
qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\sklearn\neural_netwo
rk\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (2500) reached and the optimization hasn't converged yet.
  warnings.warn(
C:\Users\Abigail Miller\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_
qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\sklearn\neural_netwo
rk\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (2500) reached and the optimization hasn't converged yet.
  warnings.warn(
C:\Users\Abigail Miller\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_
qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\sklearn\neural_netwo
rk\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (2500) reached and the optimization hasn't converged yet.
  warnings.warn(
C:\Users\Abigail Miller\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_
qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\sklearn\neural_netwo
rk\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (2500) reached and the optimization hasn't converged yet.
  warnings.warn(
C:\Users\Abigail Miller\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_
qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\sklearn\neural_netwo
rk\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (2500) reached and the optimization hasn't converged yet.
  warnings.warn(
Best Hyperparameters: {'max_iter': 2500, 'hidden_layer_sizes': [10, 10], 'activatio
n': 'logistic'}
Best R2 Value: 0.5331022357014145

```

```

In [45]: # Predicts target values for the test set using the best model from random search
y_pred = random_search.predict(X_test)
# Calculates and prints the root mean squared error (RMSE) of the predictions
print("RMSE:", mean_squared_error(y_pred, y_test)**(0.5))

```

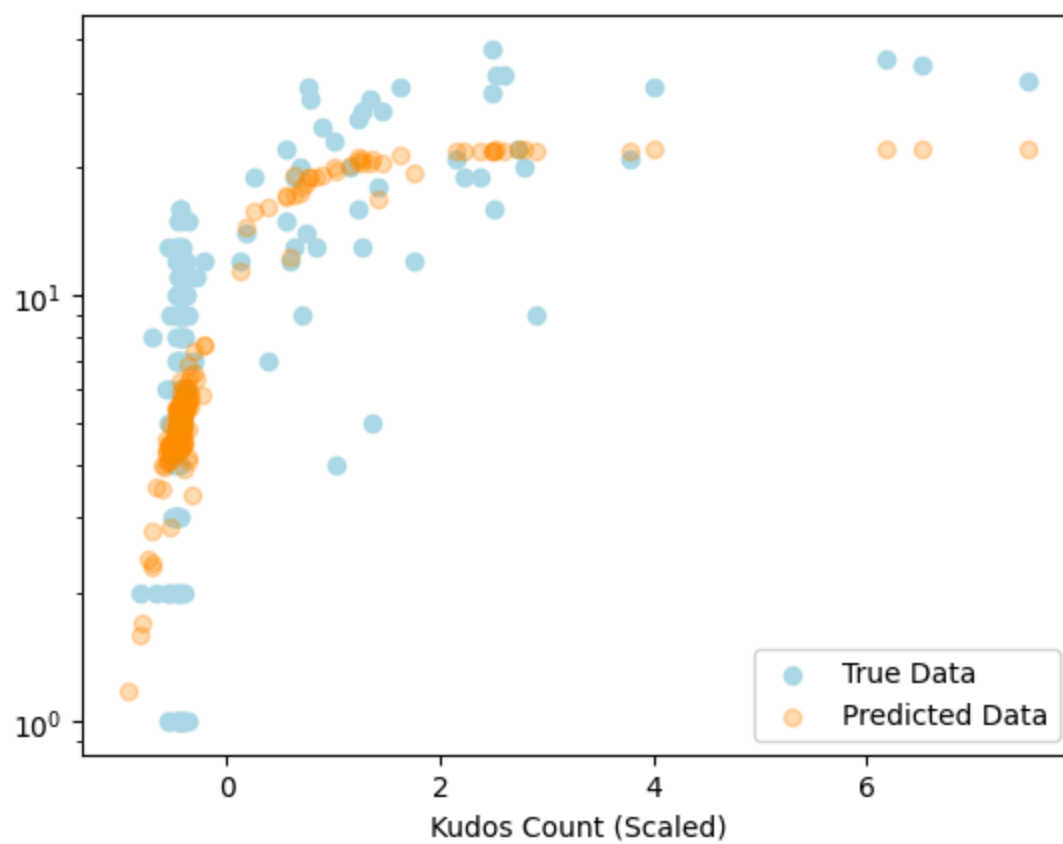
RMSE: 5.501030559325892

```

In [46]: # Creates a scatter plot comparing true vs predicted values for the test set
plt.scatter(X_test[:,0], y_test, label="True Data", color = "lightblue")
# Adds Legend and Labels
plt.scatter(X_test[:,0], y_pred, label="Predicted Data", alpha=0.3, color = "darkor
plt.legend()

```

```
plt.xlabel("Kudos Count (Scaled)")  
plt.yscale("log") # Uses logarithmic scale for y-axis  
plt.show() # Displays the plot
```



In []:

In []: